MentisDB

Durable, Verifiable Memory for Software Agents

Angel Leon

master after 0.10.10.55 (unreleased changes included) — 2026-11-01

Abstract

Agents forget. A coding agent that learned last month that the user wants tabs, that the build breaks on a certain linker flag, or that a design was rejected, starts the next session without any of it unless something durable and searchable carries it forward. MentisDB is that memory, built as an embedded Rust library and an optional single-binary daemon. Memory is an append-only log of typed records called *thoughts*. Each record is linked to its predecessor by a SHA-256 hash, can carry an Ed25519 signature bound to its chain and position, and can be governed by a write policy recorded inside the chain itself.

Retrieval combines BM25 lexical scoring with document-frequency gating, dense-vector similarity from a local ONNX model, typed graph expansion, append-order session cohesion, and optional Reciprocal Rank Fusion. Nothing is deleted: a superseded thought is replaced in results by its current successor, and one that a newer thought merely contests is shown with the contester ranked above it. A *standing context* query returns what is durably true about the user and the project regardless of age, next to the usual window of recent thoughts. None of this needs an external database, a network service or an LLM call.

On the LoCoMo-10P benchmark the default pipeline reaches 73.5% Recall@10, and on LongMemEval it reaches 68.0% Recall@5 (release 0.10.9.54, local MiniLM embeddings). On real project chains, an audit found no loss of recall with the age of a memory, but it found that near-duplicate thoughts crowd results and that paraphrased queries recall far less than verbatim ones. This paper describes the design as it is implemented, states each default exactly, and lists what is still weak.

1. The Problem and the Requirements

An agent's context window is working memory. It is erased at the end of each session and compacted during long ones. What the agent needs on top of it is the equivalent of long-term memory: a store it writes to as it works and reads from when it starts or when a question touches the past. We set eight requirements for that store.

1. **Durable.** An acknowledged write survives a process crash. With an opt-in setting it also survives power loss.
2. **Recallable regardless of age.** A preference stated three months ago must reach the agent as reliably as one stated yesterday. A window of recent records does not satisfy this on its own.
3. **Searchable by meaning and by words.** Users paraphrase. Exact identifiers, file names and error strings also matter. Both kinds of query must work.
4. **Current.** When a fact changes, the new version must win and the old one must stop surfacing, without destroying the history that explains the change.
5. **Attributable and verifiable.** Every record names the agent that wrote it. Edits to history are detectable, and a chain can require that writes carry valid signatures.
6. **Local and dependency-free.** The core path runs on a laptop with no database server, no cloud API and no per-write LLM cost.

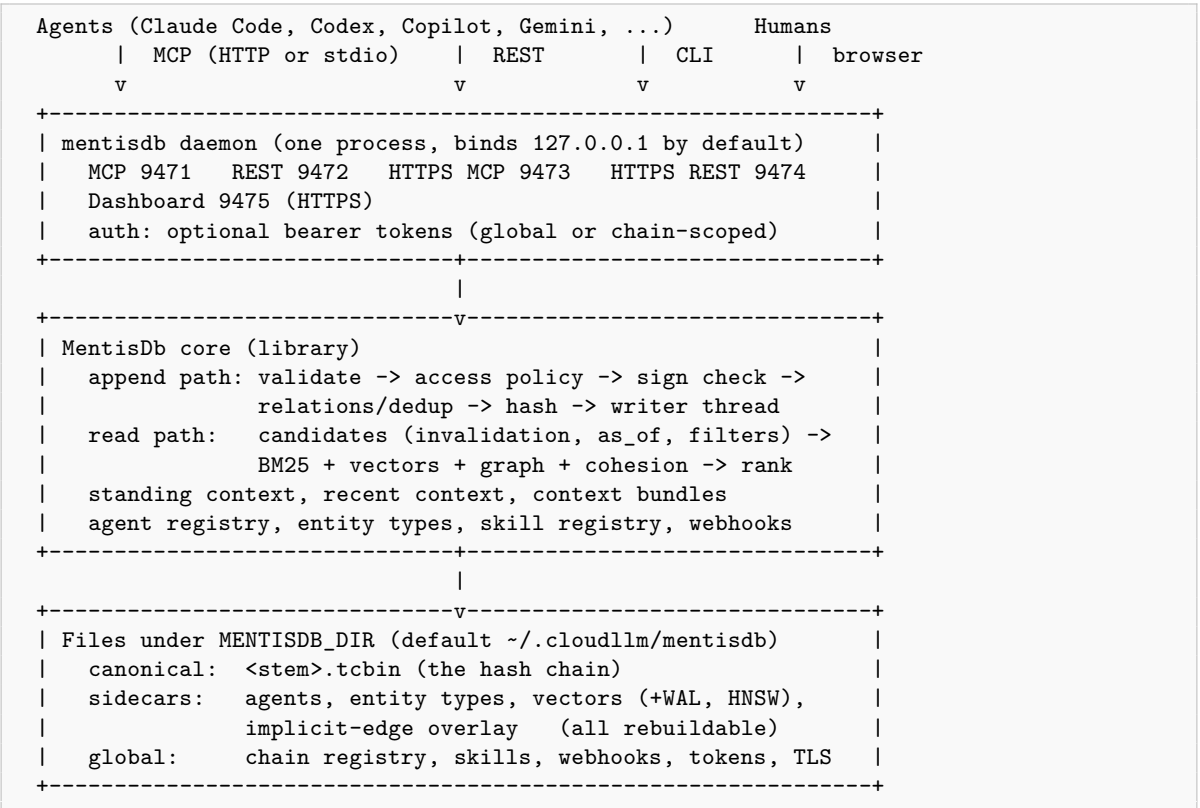
7. **Portable across agents.** Claude Code, Codex, Copilot, Gemini and any other MCP client share one memory.
8. **Inspectable.** Every ranking can be explained, and every memory can be read, exported and audited by a human.

Common approaches fail at least one requirement. A Markdown notes file is durable and portable but is not searchable at scale and is loaded whole or not at all. Provider session memory is not portable and not inspectable. A plain vector store answers paraphrases but has no notion of currency, attribution or integrity, and misses exact identifiers. Systems that extract memories with an LLM on every write add latency, cost and a second model whose mistakes become permanent.

MentisDB addresses the requirements with three ideas. Memory is an append-only, hash-chained log, so history is never rewritten and changes are recorded as new thoughts that point at old ones. Retrieval is a local, multi-signal ranking whose every component is exposed in the result. A written protocol, shipped as an agent skill, tells agents when to read, when to write and what to link.

2. Architecture

MentisDB is a Rust crate (`mentisdb`) and one binary of the same name. The crate can be embedded directly. The binary runs a daemon and also serves as the command-line client.



The canonical log is the only source of truth. Every other file is either a small registry written atomically or a derived index that can be rebuilt from the log.

Ports are set in `src/server.rs` (`MentisDbServerConfig`) and can be changed or disabled with `MENTISDB_MCP_PORT`, `MENTISDB_REST_PORT`, `MENTISDB_HTTPS_MCP_PORT`, `MENTISDB_HTTPS_REST_PORT` and `MENTISDB_DASHBOARD_PORT` (0 disables a listener). A self-signed TLS certificate is generated on first start under `tls/`.

3. Data Model

3.1 Thoughts

A thought is one memory record. The `Thought` struct (`src/lib.rs`) has these fields, in declared order. The order matters because records are encoded positionally.

Field	Type	Set by	Meaning
`schema_version`	`u32`	chain	Record layout version (currently 3)
`id`	`Uuid`	chain	Stable identifier (random v4)
`index`	`u64`	chain	Position in the chain, from 0
`timestamp`	`DateTime<Utc>`	chain	Commit time
`session_id`	`Option<Uuid>`	caller	Groups thoughts of one working session
`agent_id`	`String`	caller	Author, resolved through the agent registry
`signing_key_id`	`Option<String>`	caller	Key that signed this thought
`thought_signature`	`Option<Vec<u8>>`	caller	Detached Ed25519 signature (Section 4.2)
`thought_type`	`ThoughtType`	caller	What kind of memory this is
`role`	`ThoughtRole`	caller	How the system should treat it
`content`	`String`	caller	The memory text
`confidence`	`Option<f32>`	caller	0 to 1, clamped
`importance`	`f32`	caller	0 to 1, clamped; default 0.5
`tags`	`Vec<String>`	caller	Free-form labels, normalized; also carries the scope
`concepts`	`Vec<String>`	caller	Semantic labels, normalized
`refs`	`Vec<u64>`	caller	Back-references by index
`relations`	`Vec<ThoughtRelation>`	caller	Typed edges (Section 3.3)
`entity_type`	`Option<String>`	caller	Optional domain label, e.g. `bug_report`
`source_episode`	`Option<String>`	caller	Optional link to an external source
`prev_hash`	`String`	chain	Hash of the previous thought, empty for the first
`hash`	`String`	chain	SHA-256 of this record (Section 4.1)

The caller supplies the semantic fields. The chain assigns identity, position, time and the hash fields, so a client cannot choose where in history its record lands. For an unsigned thought the chain also normalizes input: it adds a `References` relation for each `ref`, fills a missing `valid_at` on relations with the commit time, and may add a `Supersedes` edge when automatic deduplication is on (Section 6.9). A signed thought is stored exactly as signed, with none of these additions, so its signature can be checked again later.

3.2 Types and Roles

`ThoughtType` says what kind of memory a thought is. It has 32 variants. In declared order they are:

> PreferenceUpdate, UserTrait, RelationshipUpdate, Finding, Insight, FactLearned, PatternDetected, Hypothesis, Mistake, Correction, LessonLearned, AssumptionInvalidated, Constraint, Plan, Subgoal, Decision, StrategyShift, Wonder, Question, Idea, Experiment, ActionTaken, TaskComplete, Checkpoint, StateSnapshot, Handoff, Summary, Surprise, Reframe, Goal, LLMExtracted, AccessPolicy.

`LLMExtracted` marks memories produced by the optional extraction feature. `AccessPolicy` is reserved for governance records and is refused on the normal append path.

`ThoughtRole` says how the system treats a thought: `Memory`, `WorkingMemory`, `Summary`, `Compression`, `Checkpoint`, `Handoff`, `Audit` or `Retrospective`. Type and role are independent. A context-compaction checkpoint is a `Summary` thought with role `Checkpoint`. A post-mortem lesson is a `LessonLearned` thought with role `Retrospective`.

Types drive behaviour, not only labels. Standing context (Section 6.10) selects thoughts by type.

3.3 Typed Relations

A relation is a directed, typed edge from the new thought to an earlier one, possibly in another chain:

```
struct ThoughtRelation {
    kind: ThoughtRelationKind,
    target_id: Uuid,
    chain_key: Option<String>,        // set for cross-chain edges
    valid_at: Option<DateTime<Utc>>,  // when the edge starts to hold
    invalid_at: Option<DateTime<Utc>>,// when it stops holding
}
```

There are twelve kinds: `References`, `Summarizes`, `Corrects`, `Invalidates`, `CausedBy`, `Supports`, `Contradicts`, `DerivedFrom`, `ContinuesFrom`, `BranchesFrom`, `RelatedTo` and `Supersedes`. Because edges always point backwards in time, the relation graph of a chain is acyclic by construction.

Three kinds change what retrieval shows: `Supersedes`, `Corrects` and `Invalidates`. A thought that is the target of one of them is *invalidated* (Section 3.6).

3.4 Scopes, Sessions and Entities

A thought belongs to one memory scope: `User` (the default), `Session` or `Agent`. The scope is stored as a tag (`scope:user`, `scope:session`, `scope:agent`), so it is covered by signatures and filters like any tag. User scope holds facts about the person and the project. Session and agent scopes hold material private to one session or one agent, and standing context skips them.

`session_id` groups the thoughts of one working session. `entity_type` lets a chain declare domain types such as `bug_report` or `customer`; the per-chain entity-type registry records them. Both are filters. Neither changes ranking.

3.5 Agents

Thoughts carry only an `agent_id`. Everything else about an agent lives in a per-chain registry (`AgentRecord`): display name, owner, description, aliases, status (`Active` or `Revoked`), public keys, first and last seen index and time, and a thought count. Each public key records its id, algorithm (Ed25519 is the only one), key bytes, and when it was added and revoked.

A public key is accepted only if it is a valid, non-weak Ed25519 curve point and comes with a proof of possession: a signature by that key over a payload with domain `mentisdb/key-registration/v1` that names the chain, agent, key id, key and a timestamp within ten minutes of the server clock (`KeyRegistrationProof::verify` in `src/access.rs`). Nobody can register a key they do not hold.

The registry is a sidecar file, not part of the hash chain. Section 4.3 explains what that means for trust.

3.6 Versions, Invalidation and Point-in-Time Reads

Nothing is deleted. When a thought is superseded, corrected or invalidated, the new thought carries the edge and the old one stays in the log.

At open, the chain scans all relations and builds the set of invalidated thought ids together with each target's invalidating edges (`observe_invalidating_relations`). Every append updates the set. The default read paths hide invalidated thoughts: filter search, ranked search, context bundles and recent context. A caller sees them with `include_invalidated=true`, which is how an agent audits what changed.

Hiding alone is not enough, for two reasons. A successor often uses different words than the thought it replaces, so hiding the old version can leave the agent with nothing. And agents rarely type their links. On one real installation, none of 14 `Correction` and `AssumptionInvalidated` thoughts and none of about 90 `Decision` thoughts carried a `Supersedes`, `Corrects` or `Invalidates` relation; half the corrections carried no link and the rest only plain references. The invalidated set stayed empty and the old policy kept winning searches. The lineage index (`src/lineage.rs`) addresses both problems. It is derived from the stored chain at open time, so existing chains benefit without migration.

- **Superseded.** A thought targeted by a strong edge is `superseded`. When it would have ranked, ranked search puts its *live successor* in its place: the newest unsuperseded end of the chain of supersessions, followed up to 64 steps. The successor inherits the old thought's score and reports which thoughts it stands in for (`successor_of`), so it reaches the agent even when its wording does not match the query.
- **Contested.** A weak link from a thought that says it replaces something marks the target `contested`. That covers a `Correction` or `AssumptionInvalidated` that refers to any thought, a `Decision` or `StrategyShift` that refers to an earlier decision, constraint, preference, plan or strategy with a plain reference or none, and any `Contradicts` edge. A contested thought stays visible, names the contesting thought, and that thought is promoted directly above it. Weak links do not hide, because a plain reference is often background and hiding it would lose valid memory.
- **Annotated everywhere.** Every thought returned on every surface carries `version_status` (`current`, `contested` or `superseded`) and, when not current, its `successors` and `current_version`, including results fetched with `include_invalidated=true`.
- **Advice at write time.** When a new `Correction`, `AssumptionInvalidated` or `Decision` looks like a replacement but carries no strong relation, the append response returns `warnings` naming up to three likely targets, so the agent can add the typed link.

`with_resolve_successors(false)` on a library query restores plain hiding.

A query with `as_of = T` sees the chain as it was at time `T`. It considers only thoughts committed at or before `T`, and hides a thought only if an invalidating edge was in force at `T`: the edge's source was committed by then, its `valid_at` is not after `T`, and its `invalid_at` is not at or before `T`.

The live view treats every invalidating edge as permanent. An edge whose `invalid_at` has passed still hides its target from live reads. Only `as_of` reads honour edge validity.

3.7 Chains, Branches and Merges

A chain is one independent log, named by a chain key. Keys are validated at every entry point: not blank, at most 256 bytes, no control characters (`validate_chain_key`). The chain registry (`mentisdb-registry.json`) lists chains with their schema version, storage adapter, thought and agent counts, and location.

`branch_from` creates a new chain whose first thought is a `StateSnapshot` with a cross-chain `BranchesFrom` edge to a thought of the source chain. Nothing is copied and the source is untouched; the branch inherits context by reference. Ranked search through the daemon follows `BranchesFrom` edges and also searches the ancestor chains, deduplicating by thought id. It searches

each ancestor whole, so thoughts appended to the parent after the fork point appear in the branch's results too.

Merging is a server operation. It re-appends the source chain's thoughts, unsigned, onto the target and then deregisters the source. It copies type, role, content, importance, confidence, tags and concepts, and maps agents by id. It drops relations, refs, sessions, entity types, source episodes, signatures and original timestamps, and it refuses targets that are not open-write chains. A merge therefore preserves what was remembered but not how it was connected or who signed it.

4. Integrity and Access Control

4.1 The Hash Chain

Each thought's `hash` is the lowercase hex SHA-256 of a canonical encoding of the record (`compute_thought_hash`). The canonical form is a fixed struct serialized with bincode 2 in its standard configuration. It contains every field except `hash` itself and two fields excluded for backward compatibility: `entity_type` and `source_episode`. It includes `prev_hash` and the signature fields. Every thought therefore satisfies:

```
thought.hash      == sha256(bincode(canonical(thought)))
thought.prev_hash == previous_thought.hash      ("" for index 0)
thought.index     == its position
```

Every open verifies the whole chain (`verify_integrity`): it checks each index, each `prev_hash` link, and recomputes each hash. This costs O(n) and runs before the chain serves reads. Checking links alone would miss an edited record, which is why the hashes are recomputed.

This is tamper *evidence*. An attacker who can write the file can rewrite it entirely and recompute every hash. Evidence becomes useful when the head hash is recorded somewhere the attacker cannot reach, and signatures (Section 4.2) bind individual records to keys the attacker does not hold.

Two gaps are known. First, `entity_type` and `source_episode` are not covered by the record hash, so they can be edited in the file without breaking the chain (v2 signatures do cover them). Second, migrating a chain from schema V0 to V2 recomputes all hashes without checking the old ones, so tamper evidence does not extend across that migration.

4.2 Signatures

A thought may carry a detached Ed25519 signature (RFC 8032, strict verification) and the id of the signing key. The signature is checked on append against an active, unrevoked key of the thought's agent, on every chain. A bad signature is always refused, whatever the chain's policy.

Two payload versions exist, distinguished by the stored signature itself:

Stored `thought_signature`	Version	Payload
64 bytes	v1	Legacy JSON of the semantic fields. Not bound to a chain or position, so replayable; accepted only on open chains.
65 bytes starting `0x02`	v2	Domain `mentisdb/thought/v2`, then chain key, agent, `prev_hash`, session, type, role, content, confidence, importance, tags (which include the scope), concepts, refs, relations with validity, entity type and source episode.

The v2 payload (`encode_thought_v2` in `src/signing.rs`) uses big-endian length-prefixed fields and a NUL-terminated domain tag, so a signature for one payload type can never verify as another. Binding the chain key and the previous head means a signed thought is valid at exactly one position of one chain. The cost is a race: if another writer moves the head first, the append fails with the typed error `StaleHead` and the client signs again against the new head. The `id`, `index` and `timestamp` are assigned by the chain and covered by the record hash rather than the signature.

Signing happens on the client. The CLI signs locally with `--private-key-file` and never sends the seed.

4.3 Write Policies Recorded in the Chain

Each chain has a write policy (`AccessMode` in `src/access.rs`):

Mode	A new thought must
`Open` (default)	nothing more; unsigned writes are accepted
`Signed`	carry a valid v2 signature by an active key registered to its agent
`Allowlist`	carry a valid v2 signature by a key in the policy's allowed set that is still active in the registry

The policy is not configuration. It lives in the chain as signed `AccessPolicy` thoughts, replayed at open. Each record holds the mode, the governor key set, an RFC 6962 Merkle root over that set, a sequence number and the hash of the previous policy record. A change after the first record must be signed by a current governor. The first record is trust-on-first-use: it is accepted only from an operator-authenticated entry point (the library, a global bearer token, or the dashboard PIN), and its signer becomes the first governor. An invalid record found on replay is ignored for enforcement and reported by `mentisdb chain access verify`.

`Signed` chains trust the agent registry's key list, which is a sidecar outside the hash chain. `Allowlist` chains do not, because their allowed keys are committed in the chain. Policies govern writes only; they do not restrict reads. The full design, formats and threat model are in `docs/CHAIN_ACCESS.md`.

4.4 Transport Authentication

The daemon binds to `127.0.0.1` by default. Bearer tokens are optional and off by default (`src/auth.rs`). When enabled, tokens are stored as SHA-256 hashes in a `0600` file and compared in constant time. A token is either global or scoped to one or more chains. REST and MCP share one table that marks each operation as chain-scoped or global-only, normalize chain keys before authorization, and authorize the same fields the handler executes. Chain-scoped tools that name no chain require access to the default chain.

While tokens are off, requests that come from a browser (they carry `Origin` or `Sec-Fetch-Site`) must name an allowed host, which blocks DNS-rebinding and cross-site requests to the loopback listeners. The dashboard sits behind an optional PIN with a per-IP limit of five failures per five minutes, sends a strict Content-Security-Policy and frame-denial headers, and guards state-changing calls against cross-site requests.

Several authorization bypasses have been found and fixed, in 0.10.9 and again in the unreleased changes, including decoy chain-key fields that let a chain-scoped token read another chain and a fail-open default for unlisted MCP tools. Deployments that expose the daemon beyond loopback should run current master and enable tokens.

5. Storage and Durability

5.1 Record Format

Each chain is one append-only file, `<stem>.tcbin`. Each record is an 8-byte little-endian length followed by the bincode 2 encoding of the `Thought` (`persist_binary_thought`). A record payload may not exceed 10 MiB.

The file stem is the chain key with characters outside `[A-Za-z0-9_-]` replaced by `_`, a dash, and the first 16 hex characters of the SHA-256 of the chain key. Two keys that sanitize to the same string therefore still get different files.

A legacy JSON-lines format (`.jsonl`) can still be read for migration but no new chain uses it.

5.2 File Layout

```
$MENTISDB_DIR/                          (default ~/.cloudllm/mentisdb)
  mentisdb-registry.json                chain registry
  mentisdb-skills.bin                   skill registry
  mentisdb-webhooks.json                webhook registrations
  bearer-tokens.json                    hashed bearer tokens (0600)
  tls/cert.pem, tls/key.pem             self-signed certificate
  .fastembed_cache/                     local embedding model
  <stem>.tcbin                          the hash chain (canonical)
  <stem>.agents.json                    agent registry
  <stem>.entity-types.json              entity-type registry
  <stem>.vectors.<model>.<ver>.<dim>d.json       vector sidecar snapshot
  <stem>.vectors.<model>.<ver>.<dim>d.json.wal   its write-ahead log
  <stem>.vectors.<model>.<ver>.<dim>d.hnsw.bin   persisted HNSW graph
  <stem>.vectors.managed.json           which sidecars stay in sync on append
  <stem>.auto_edges.bin                 implicit-edge overlay
  migrations/vX_to_vY/                  archived pre-migration files
```

5.3 Write Path and Durability Modes

Appends go through a dedicated writer thread per chain. Two modes exist:

Mode	Behaviour	What a crash can lose
Strict (`auto_flush = true`, the default)	Each append returns after its group commit is written to the operating system. When other appends are already queued, the writer waits up to 2 ms (`MENTISDB_GROUP_COMMIT_MS`) to batch them.	Nothing on a process crash. On power loss, writes since the last sync unless `MENTISDB_FSYNC=1`.
Buffered (`auto_flush = false`)	Appends are queued (capacity 128) and written every 16 records.	Queued and unwritten records.

`MENTISDB_FSYNC=1` adds one `fsync` per group commit, and registry files are then synced before their atomic rename. Explicit flush, shutdown and drop always sync. A failed group commit is rolled back to the pre-commit file length, so a full disk cannot leave a torn record followed by its duplicate.

One writer process per data directory is assumed. There is no cross-process file lock.

5.4 Crash Recovery

Opening a chain repairs only what is safe to repair.

- **Torn tail.** If the final record is strictly partial (a short length prefix, or a payload that ends at end of file), it is copied to `<file>.torn-<millis>` with mode `0600` and truncated before the writer reopens the file.
- **Mid-file corruption.** If the bytes after a bad length prefix still decode as a complete record with a 64-digit hex hash, the damage is not a torn tail. The open fails with `InvalidData` and the file is left untouched, so no acknowledged thought is ever discarded as "tail".
- **Bounded decoding.** Every decode of untrusted bytes runs under a budget: 64 MiB per thought record and 1 GiB per derived file (`src/decode_limits.rs`). A forged length prefix yields `InvalidData` instead of an allocation that aborts the process.

These rules are pinned by adversarial tests that cut the last record at every byte (`tests/core_crash_recovery_tests.rs`).

5.5 Derived State

Derived files make search fast, and each one can be rebuilt from the log.

- **Vector sidecars** store one embedding per thought for one model, version and dimension. Changing any of these selects a different file, so incompatible embeddings never mix. Each append writes one integrity-chained record to the sidecar's write-ahead log (magic `MDBVWAL1`). The snapshot is compacted when the log reaches one sixteenth of the entries, and at least 32. A sidecar is fresh only if its thought count, head hash and entry count all match the chain.
- **Background rebuild.** Opening a chain never embeds its history on the request path. A missing or stale sidecar is queued to one process-wide worker, one job per sidecar at a time, built in batches of 32 with one shared model, and adopted when done. Until then ranked search serves lexical and graph signals. Chains of 32 thoughts or fewer embed inline.
- **Implicit-edge overlay** (Section 6.6) records the thought count and head hash it covers. On load it replays a missing tail of up to 2048 thoughts, and otherwise rebuilds.
- **HNSW graphs** are reloaded only when their stored count matches the chain.

5.6 Schema Versions and Migration

Version	Change
V0	Original layout, no version field (JSON lines)
V1	Explicit version, registry-backed chains, optional signatures
V2	`Reframe` type, `Supersedes` relation, cross-chain `chain_key` on relations
V3	`valid_at` and `invalid_at` on relations; later `entity_type` and `source_episode` were appended without a version bump

The loader tries the current layout first and falls back record by record to the legacy layouts, so a file with mixed versions still loads. When the daemon starts it migrates registered chains before serving: it writes the new file beside the old one, archives the old file under `migrations/`, and renames the new file into place. A chain written with the older JSON-based hash is rehashed with the bincode hash on open. Both steps skip work that is already done, so running them twice is harmless. Both also block: a large chain is unavailable until its migration finishes.

Because bincode encodes enum variants by position, new variants must be appended at the end

of an enum and existing ones never reordered. A chain that contains a variant added later, such as an `AccessPolicy` record, cannot be read by an older MentisDB.

5.7 Backups

A backup is a ZIP file named `mentisdb-YYYY-MM-DD-HH-MM-SS.mentis` holding a manifest with the SHA-256 and size of each file, the registries, chain files and sidecars, skills, webhooks and optionally the TLS material. Restore checks the manifest before writing, bounds sizes and file counts, rejects symlinks and zip bombs, and never splices raw records into a chain with a `Signed` or `Allowlist` policy; such a backup is imported as a separate, renamed chain.

6. Retrieval

Two read paths exist. *Filter search* (`query`) returns thoughts that match indexed fields and a case-insensitive substring, in append order. *Ranked search* (`query_ranked`) scores candidates with several signals and returns them best first. Agents use ranked search almost always. This section describes ranked search with its defaults as of current master.

6.1 Candidates

Ranking starts from a candidate set (`retrieval_candidates`). It removes invalidated thoughts unless `include_invalidated` is set, applies `as_of` (Section 3.6), and applies filters on type, role, agent, tags, concepts, scope, entity type, session and time range. Filters only restrict; they never add score. After scoring, superseded thoughts that would have ranked are replaced by their live successors and contested thoughts are preceded by their contesters (Section 3.6).

6.2 Lexical Signal: BM25 with Document-Frequency Gating

Text is normalized by lowercasing, tokenizing and stemming with the Snowball English (Porter2) stemmer. At query time, a table of 325 irregular forms maps words such as “went” and “saw” to their lemmas (“go”, “see”), which stemming cannot do.

Each thought is indexed in five fields, scored with BM25 (`k1 = 1.2`, `b = 0.75`) and weighted:

Field	Weight	DF cutoff
content	1.0	30%
tags	1.6	30%
concepts	1.4	30%
agent id	1.5	70%
agent registry text (name, owner, description, aliases)	1.1	60%

```
idf(t)        = ln( (N - df + 0.5) / (df + 0.5) + 1 )
score(t, f)   = idf(t) * tf*(k1+1) / (tf + k1*(1 - b + b*len_f/avg_len_f)) * weight_f
```

MentisDB has no stop-word list. Instead, for each query term and each field, the term is skipped in that field when `df / N` exceeds the field’s cutoff. This *document-frequency gate* is a hard cliff rather than a down-weight: a term in 29% of thoughts scores fully in `content`, and one in 31% scores nothing there. A gated term can still score through a field whose cutoff it respects. Details that matter in practice:

- By default `df` counts thoughts that contain the term in *any* field (global scope), and that one ratio is compared with each field’s cutoff. Registry text is attached to every thought

of an agent, so a word in an agent's description counts as present in all of that agent's thoughts. On one real 750-thought chain, project words that occur in 2% to 9% of the content were gated out of it by registry text alone.
- A per-field scope that counts each field separately exists as an opt-in (`MENTISDB_DF_SCOPE=field`, or `df_scope` on the library query). It is not the default because it has not yet been run through LoCoMo and LongMemEval.
- Gating is off for chains under 20 thoughts. `N` counts every committed thought, including invalidated ones and ones excluded by filters. The IDF of a surviving term always uses the global `df`.

Synonyms. The daemon expands query terms with a built-in thesaurus of 895 headwords, keyed by stem so inflected forms expand too. Each term gets at most three synonyms at weight 0.3 (`apply_thesaurus_if_text` in `src/server.rs`). The library does not expand by default; embedders pass synonyms explicitly.

6.3 Vector Signal and Fusion

The default build includes local embeddings through fastembed: the `all-MiniLM-L6-v2` model, 384 dimensions, run on ONNX on the CPU. A dependency-free `local-text` provider (256 dimensions) is the fallback. `MENTISDB_EMBEDDINGS` selects `auto` (default), `fastembed`, `local-text` or `off`.

The query is embedded and compared by cosine with each thought's vector. Up to 256 hits per sidecar are kept, and hits under cosine 0.04 are dropped. The vector score is `0.5 * cosine`. It is fused with the lexical score by:

```
vector_contribution = vector * (1 + 35 * exp(-lexical / 3))
```

With no lexical match the vector signal is amplified 36 times, so a paraphrase with no shared words can still rank. As the lexical score grows the amplification decays smoothly, so vectors help moderate lexical matches without overriding strong ones.

Below 50,000 vectors the search is an exact scan. At or above that threshold (`MENTISDB_HNSW_THRESHOLD`) it switches to an HNSW graph (M 48, ef_construction 400, ef_search 128). A filtered HNSW search scans exactly when the allowed set is small (at most 4096 thoughts or under 5% of the corpus) and otherwise oversamples, so filters do not lose matches the graph skipped.

6.4 Graph Expansion

Graph expansion is off unless the query sets `graph`. When on, the top 20 lexical hits seed a breadth-first traversal over explicit relations and the implicit-edge overlay, bidirectional by default, to depth 2, visiting at most 128 thoughts (`expand_with_overlay_classified`). Each reached thought gets three scores:

- `graph = 1 / depth`,
- `seed_support = 0.2 * min(paths - 1, 5)` for thoughts reached from several seeds,
- `relation = boost(kind) / depth` for the strongest edge kind on its path.

Relation kind	Boost
ContinuesFrom	0.60
BranchesFrom	0.55
Corrects, Invalidates	0.50
Supersedes	0.45
DerivedFrom	0.40
Summarizes, CausedBy	0.20

Relation kind	Boost
Supports, Contradicts	0.15
RelatedTo (implicit)	0.08
References	0.06

6.5 Session Cohesion, Importance, Confidence and Recency

Session cohesion surfaces the turns around a good match, which often hold the answer without sharing its words. Every candidate with a lexical score of at least 3.0 is a seed. A candidate whose own lexical score is under 5.0 and that lies within 12 positions of a seed receives `1.2 * (1 - d / 12)`, where `d` is the distance in append order. This is always on (`apply_session_cohesion`).

Importance shifts close races towards what the author marked as important: `lexical * ( importance - 0.5) * 0.3` when there is a lexical match, else `(importance - 0.5) * 0.1`. **Confidence** adds `0.1 * confidence`. **Recency** adds at most 0.05, linear in append position; there is no time-based decay. Recency is deliberately a tie-breaker so that old memories are not pushed down.

The total is:

```
total = lexical + vector_contribution + graph + relation + seed_support
      + importance + confidence + recency + session_cohesion (+ rrf)
```

6.6 Implicit Edges

Most agents write few explicit relations, which leaves graph expansion little to walk. The implicit-edge overlay adds `RelatedTo` edges between thoughts whose embeddings have cosine at least 0.85, keeping the 5 strongest per thought (`MENTISDB_AUTO_EDGE_THRESHOLD` , `MENTISDB_AUTO_EDGE_K`). Up to 128 thoughts the build compares all pairs. Above that it runs one nearest-neighbour search per thought through the vector backend. Each append adds the new thought's edges with one linear scan. Chains above 50,000 thoughts get no overlay. The overlay is saved every 32 appends and is never part of the hash chain.

6.7 Optional Reciprocal Rank Fusion

With `enable_reranking` , the top 50 candidates are ranked three times: by lexical score, by vector score, and by graph-related score. Only candidates with a positive score enter each list. Each candidate then gains a consensus bonus:

```
rrf = 50 * sum over lists containing it of 1 / (60 + rank)
```

A candidate ranked first in all three lists gains about 2.5. RRF is off by default. Before 0.10.9 it replaced the additive total and collapsed rankings; it now only adds.

6.8 Explaining a Ranking

Each hit carries its full score breakdown: `lexical` , `vector` , `graph` , `relation` , `seed_support` , `importance` , `confidence` , `recency` , `session_cohesion` , `rrf` and `total` , plus the matched terms, the fields they matched in, and for graph hits the distance, seed path count, relation kinds and path. A human or an agent can see why any thought ranked where it did.

Context bundles (`query_context_bundles`) group results by seed: each lexical seed comes with the thoughts graph expansion reached from it and the path that reached them, so an agent can read

supporting memories in provenance order.

6.9 Deduplication on Write

Automatic deduplication is off unless `MENTISDB_DEDUP_THRESHOLD` is set; 0.85 is a suggested value, not a default. When on, each unsigned append compares the new thought's normalized token set with the last 64 thoughts (`MENTISDB_DEDUP_SCAN_WINDOW`) by Jaccard similarity. If the best match reaches the threshold, the new thought gets a `Supersedes` edge to it, which hides the older one by default. A prior thought the author rated more important than the new one is never superseded, so a throwaway restatement cannot hide a critical constraint. Signed thoughts are never deduplicated.

The window is a real limit: a fact re-recorded hundreds of thoughts later is not caught. Section 9.4 shows that duplicates are a measured problem on real chains.

6.10 Standing Context and Recent Context

Memory has two halves. The episodic half is what happened, in order. The semantic half is what stays true. Recent context (`mentisdb_recent_context` , 12 thoughts by default) serves the first. It forgets: a preference from three months ago has scrolled out of any practical window.

Standing context (`MentisDb::standing_context` , `src/standing.rs`) serves the second. It selects live thoughts of the standing types, ranks them by importance and then recency, collapses restatements, and caps the result:

Group	Types	Cap
Constraint, Preference, User trait, Relationship, Goal, Decision, Lesson	`Constraint` , `PreferenceUpdate` , `UserTrait` , `RelationshipUpdate` , `Goal` , `Decision` , `LessonLearned`	8 per type, 40 in total
Handoff	`Handoff`	the newest one
Checkpoint	`Checkpoint` or `Summary` type or role	the newest one

Only user-scoped or unscoped thoughts are standing. Within a group, restatements with Jaccard similarity of 0.85 or more collapse to the newest. A superseded constraint or decision is replaced by its live successor, even when the successor is not itself a standing type such as a `Correction` , and contested entries are flagged in the prompt. The result is a compact prompt block delimited by `=== STANDING CONTEXT ===` that the agent loads at startup, next to the recent-context block. It is available as an MCP tool, a REST route (`POST /v1/standing-context`) and a CLI command (`mentisdb thought standing`).

6.11 What Is Not in the Default Path

The crate contains components that ranked search does not use by default, and this paper makes no claim based on them: personalized PageRank (`src/search/ppr.rs`), pseudo-relevance feedback (`query_expansion.rs`), a query-intent classifier (`query_intent.rs`), embedding synonyms, and an `llm_rerank` flag that nothing reads. LLM memory extraction (`src/llm.rs`) exists as an explicit, opt-in operation that writes `LLMExtracted` thoughts.

7. The Agent Protocol

A memory store is only as good as the habits of the agents that use it. MentisDB ships its operating protocol as a skill (`MENTISDB_SKILL.md`), served to MCP clients as the resource

`mentisdb://skill/core` and pointed to by the server's `initialize` instructions. Its main rules are these.

Startup. List chains and pick the project's chain, bootstrap it (which writes a genesis checkpoint only on an empty chain), load the skill, list agents and reuse an existing identity, load standing context, load recent context, then write a checkpoint. Agents never invent a new agent id; they ask the user.

Search before write. Before every append the agent runs a ranked search for its proposed memory and makes three decisions. Should it write at all, or is this already known? Should it correct, invalidate or supersede an earlier thought instead of writing a parallel one? Which one to three earlier thoughts should it link to, with which typed edge? This keeps chains small and connected, and it is the main defence against duplicates. Automatic deduplication is only the downstream half. A change of policy or decision must carry a `Supersedes`, `Corrects` or `Invalidates` relation to the thought it replaces. A plain reference only marks the old version contested, and if the append response carries warnings the agent adds the typed link.

Write triggers. Write immediately on a lesson learned, a decision, a constraint, a correction or invalidated assumption, a checkpoint, a completed task, an open question or a hypothesis. One strong memory is better than many weak ones.

Recall triggers. Search before answering when a question touches the user's preferences, a past decision, a constraint or how something was done before, or when the user says "again", "remember" or "last time". The agent checks each hit's `version_status` and answers from the current version when a hit is contested or superseded. Searching with `include_invalidated=true` is for history, not for finding the current answer. Never say "I don't know your preference" without having searched.

Standing context covers what must be known at the start. Recall triggers cover what becomes relevant later. Together they address the complaint that users have to repeat themselves.

8. Interfaces

MCP. The daemon serves the Model Context Protocol over streamable HTTP. A stdio mode lets clients that only speak stdio connect through a local proxy that starts the daemon if needed. The server exposes 48 tools covering bootstrap, append and retrospectives, filter and ranked search, standing and recent context, context bundles, traversal, federated search across chains, branches and merges, agents and keys, access policies and signature verification, entity types, import and export, vectors, webhooks and the skill registry.

REST. A versioned API under `/v1/` mirrors the tools with 51 routes and can be embedded in another server with `rest_router`.

CLI. The `mentisdb` binary is also a client for every MCP tool, REST route and dashboard action, with typed argument validation and `--help` examples on each command (`docs/CLI_PARITY.md` maps each one and a test keeps the map in sync). For example:

```
mentisdb thought append "Use Postgres for the billing service" \
  --chain myproj --agent dev --type Decision
mentisdb search "which database did we pick for billing" --chain myproj
mentisdb thought standing --chain myproj
mentisdb chain access verify --chain myproj
```

Dashboard. A local HTTPS web interface browses chains, thoughts, agents, skills and access policies, and manages settings behind its PIN.

Skill registry. Agent instruction bundles are versioned in `mentisdb-skills.bin`. The first version is stored whole and later versions as diffs. A skill can be deprecated or revoked with its history kept, or deleted permanently. An agent with registered keys must sign its uploads. The registry is integrity-checked as a file; it is not a hash chain.

Webhooks and setup. Webhooks notify external services of appends. `mentisdb setup` configures MCP clients: Claude Code, Claude Desktop, Codex, Copilot CLI, VS Code Copilot, Gemini, Grok, OpenCode and Qwen.

9. Evaluation

9.1 Setup

Two public benchmarks measure retrieval recall: for each question, does the gold evidence turn appear in the top k thoughts? Both harnesses are in the repository (`locomo-benches/`, `lme-benches/`) and run against a live daemon with a fresh chain.

- **LoCoMo-10P**: ten long multi-session conversations, 1977 questions, split into single-hop (1554) and multi-hop (423). Turns are ingested with `ContinuesFrom` edges between consecutive turns, speaker and turn tags, and importance 0.8 or 0.2 by speaker. Queries use ranked search with graph expansion to depth 3. A hit is substring containment of the gold evidence in a returned thought.
- **LongMemEval**: 500 questions over a chain of about 10,900 turns, scored at R@5, R@10 and R@20.

Both use the default embedding model (`fastembed` MiniLM, 384 dimensions) and the daemon's automatic thesaurus.

9.2 Results

Release 0.10.9.54 (October 2026) is the latest full run:

Benchmark	Pipeline	R@5	R@10	R@20
LoCoMo-10P	default	63.9%	73.5%	
LoCoMo-10P	with RRF	66.0%	74.1%	
LongMemEval	default	68.0%	72.8%	78.8%
LongMemEval	with RRF	68.6%		

Query throughput was about 384 queries per second on both. The changes after 0.10.9.54 up to the date of this paper do not touch ranking defaults, except where Section 6 says so.

Earlier full runs show where the gains came from:

Release	Main change	LoCoMo-10P R@10	LongMemEval R@5
0.8.0	baseline		57.2%
0.8.0	Porter stemming		61.6%
0.8.1	smooth fusion, cohesion, DF gate	74.2%	67.6%
0.8.9	irregular lemmas	71.9%	66.8%
0.9.9	automatic thesaurus, real embeddings	72.6%	66.8%
0.10.9.54	stemmed thesaurus keys, damped synonyms, RRF fix	73.5%	68.0%

On a two-conversation LoCoMo subset, 0.8.1 reached 88.7% R@10. That number is for the subset only and should not be compared with full-benchmark results.

9.3 Where Misses Come From

A near-miss analysis of 0.8.5 on LoCoMo-10P bucketed the misses at R@10. A quarter of them (25.8%) were found by R@20, which is a ranking error. Of the rest, 43.3% were not found even by R@50, because the evidence shares almost no words with the question. A similar analysis of 0.9.8 on LongMemEval found that 65.7% of misses were not in the top 20. Multi-hop questions remain the weakest category on LoCoMo, about 17 to 20 points below single-hop. The lesson is that lexical retrieval has a hard ceiling on paraphrase, and a small embedding model only partly lifts it.

9.4 Recall on Real Project Chains

Benchmarks use conversations, not the memories coding agents actually write. The `examples/recall_audit.rs` tool audits a copy of a real chain. It builds synthetic queries from the chain's own thoughts, runs them through the production ranked search with the daemon's defaults, and reports where each target lands, bucketed by its age. Query kinds range from easy (`drop50`: the target's words with half removed) to hard (`ref`: the opening of a later thought that refers to the target, and `sem`: the opening of the target's nearest semantic neighbour among thoughts sharing few of its words).

The audit of real chains found three things:

1. **No age effect.** Recall did not fall with the age of the target. Older memories are found as reliably as recent ones once the query reaches them. Ranking does not bury the past; startup loading did, which standing context addresses.
2. **Duplicates crowd results.** Near-duplicate thoughts, written because agents did not search first or because a fact was restated outside the dedup window, took several of the top ten slots and pushed other relevant memories out.
3. **Paraphrase is weak.** Paraphrase-style queries recalled the target in the top ten only 43% to 55% of the time, against much higher recall for verbatim-style queries.

These findings set the priorities in Section 10.

9.5 Cost

Operation	Measurement
Unsigned append, open chain (buffered, CPU only)	about 55 µs
Signed append (one Ed25519 verification)	about 75 to 80 µs
Append with a 3000-thought managed sidecar	2.1 ms
Rare one-term lexical query, 100,000 documents	1.8 µs
Three-term lexical query, 100,000 documents	0.93 ms
100 vector queries, 10,000 x 128 dimensions, exact scan	33.8 ms (recall@10 100%)
100 vector queries, 10,000 x 128 dimensions, HNSW	8.5 ms (recall@10 91.4%)

The first five figures are from the Criterion benches `access_policy` and `storage_paths` on Apple silicon in release mode, as recorded in the changelog and `docs/CHAIN_ACCESS.md`. The vector figures are from `hnsw_comparison`, re-run for this paper on an Apple M4 Max with synthetic unit vectors. HNSW is about four times faster here; it only becomes the default backend at 50,000 vectors, where an exact scan grows linearly. With `MENTISDB_FSYNC=1` the disk flush dominates append cost.

10. Limitations and Future Work

10.1 Limitations

- **Paraphrase recall.** On real chains, paraphrased queries find the target in the top ten only about half the time. The default encoder is a small model whose cosine scores are compressed, and its training window is shorter than long thoughts.
- **Duplicates.** Deduplication is off by default and sees only the last 64 thoughts. Restated facts accumulate and crowd results. The protocol's search-before-write rule is the main defence, and it depends on agent discipline.
- **Global document frequency.** The default gate counts registry text as present in every thought of an agent, which removes useful project words from content scoring. The per-field alternative is not yet benchmarked.
- **Currency depends on links.** Only typed `Supersedes`, `Corrects` and `Invalidates` edges hide an old version. Untyped replacements are detected by a type-based rule and only marked contested, and a replacement with no link at all is caught only by the append-time warning.
- **Live view ignores edge expiry.** An invalidating edge hides its target forever in live reads, even after its `invalid_at`.
- **Integrity gaps.** Two fields are outside the record hash, legacy migration rehashes without checking, and the agent registry is outside the chain, so `Signed` chains trust a sidecar. Integrity is local: there is no consensus and no external anchoring of head hashes.
- **Reads are not access-controlled.** Write policies do not restrict who can read a chain. Bearer tokens are the only read control and are off by default.
- **Blocking migrations.** Schema and hash migrations run before a chain serves and rewrite the whole file.
- **Merges lose structure.** A merge drops relations, signatures and timestamps.
- **Single writer.** Two processes writing one data directory are not detected.
- **Benchmarks measure retrieval, not answers.** Recall at k says whether the evidence was retrieved, not whether the agent used it well.

10.2 Future Work

- **Duplicate collapse in results:** fold near-duplicate clusters into one hit so that restatements stop crowding the top ten.
- **Measuring valid-version retrieval** on benchmarks with changing facts, and on real chains with the correction audit (`examples/correction_audit.rs`).
- **Better paraphrase recall:** retune vector fusion, evaluate larger local encoders, and embed long thoughts in chunks.
- **Per-field document frequency** as the default once benchmarked.
- **Read access control:** private chains whose reads require authentication, with mode changes recorded in the chain, and key administration restricted to the operator.
- **Non-blocking, recoverable migrations** and hash coverage of every field.
- **Anchoring head hashes** externally so that a full rewrite of a chain becomes detectable.
- **Answer-level evaluation** with an LLM reader, alongside recall.

11. Related Work

Several systems give agents long-term memory. Mem0 extracts and updates memories with an LLM over a vector store. Graphiti, behind Zep, builds a temporal knowledge graph with LLM extraction and validity intervals. Letta, formerly MemGPT, lets the model page memory in and out of its own context. These systems put an LLM on the write path, which gives them structured facts at the cost of latency, money and a second model's errors.

MentisDB makes the opposite trade. It keeps what the agent wrote, verbatim and typed, adds no model to the write path, and puts the work into retrieval, the write protocol and integrity. Its distinguishing properties are the hash chain, signatures bound to chain position, write policies governed inside the chain, standing context, and fully decomposed scores, all in one embedded binary. Validity intervals on relations are shared with Graphiti's design. Hybrid lexical and vector retrieval is common to most systems.

12. Conclusion

MentisDB treats agent memory as a verifiable log of typed thoughts and builds retrieval, currency and governance on top of it without an external service. Its defaults are stated in this paper exactly, its benchmark numbers come with their setup, and its weaknesses are measured: paraphrase recall and duplicates, not age, are what limit recall on real chains. That is where the next work goes.

References

- Robertson, S. and Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval, 3(4), 2009.
- Cormack, G. V., Clarke, C. L. A. and Büttcher, S. *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods.* SIGIR, 2009.
- Porter, M. F. *An algorithm for suffix stripping.* Program, 14(3), 1980.
- Jaccard, P. *Étude comparative de la distribution florale dans une portion des Alpes et des Jura.* Bulletin de la Société Vaudoise des Sciences Naturelles, 1901.
- Malkov, Y. A. and Yashunin, D. A. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs.* IEEE TPAMI, 2018.
- Bernstein, D. J., Duif, N., Lange, T., Schwabe, P. and Yang, B.-Y. *High-speed high-security signatures.* Journal of Cryptographic Engineering, 2012.
- Josefsson, S. and Liusvaara, I. *Edwards-Curve Digital Signature Algorithm (EdDSA).* RFC 8032, 2017.
- Laurie, B., Langley, A. and Kasper, E. *Certificate Transparency.* RFC 6962, 2013.
- FIPS PUB 180-4. *Secure Hash Standard (SHS).* NIST, 2015.
- Reimers, N. and Gurevych, I. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP, 2019.
- Maharana, A. et al. *Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo).* ACL, 2024.
- Wu, D. et al. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.* 2024.
- Anthropic. *Model Context Protocol (MCP) Specification.* 2024.